



MCP-POWERED TOOL-AGENT ARCHITECTURE

The Agentic RAG Engine

Architecture, Retrieval Pipeline & the Model Context Protocol Tool-Agent Layer

LangGraph · Model Context Protocol (official `mcp` SDK) · BM25 + FAISS + RRF + Cross-Encoder Retrieval

Technical Reference & Codebase Deep-Dive

Covering the retrieval core, the LangGraph multi-agent graph, the standalone MCP server & client, dynamic tool routing, and resilience design

Core Framework

LangGraph state machine

Protocol

Model Context
Protocol (stdio)

Retrieval

Hybrid BM25 + FAISS
+ RRF + Reranker

Table of Contents

1	Introduction	1
1.1	Design Philosophy	1
2	High-Level Architecture	2
3	The Retrieval Pipeline	3
3.1	retriever.py — Corpus Discovery	3
3.2	chunking.py — Fixed-Window Chunking	4
3.3	embeddings.py — Dense Vectors	4
3.4	Faiss_Search.py — Dense Nearest-Neighbour Index	4
3.5	Hybrid_Search.py — Sparse Search & Reranking	5
3.6	llm_calling.py — Orchestration & the Shared Pipeline	5
4	The LangGraph Multi-Agent System	7
4.1	State Schema	7
4.2	Node: planner	7
4.3	Node: tool_agent — the Tool Agent	8
4.4	Node: critic_agent	8
4.5	Node: router (conditional edge)	9
4.6	Node: Synthesizer	9
4.7	Graph Assembly	9
4.8	The System Prompts	11
5	The MCP Layer: Server & Client	11
5.1	Process Architecture	11
5.2	mcp_server.py — the standalone server	12
5.3	Corpus-boundary enforcement: retrieve_file	13
5.4	MCP Tool Specification	14
5.5	mcp_client.py — launching and talking to the server	14
6	The Tool Agent Layer: Autonomous Selection, State & Resilience	15
6.1	Pillar 1 — Autonomous Tool Selection	15
6.2	Pillar 2 — State Alignment	16
6.3	Pillar 3 — Graceful Degradation & Schema Fallbacks	16
6.4	Pillar 4 — Preparation for Step 5 (Human-in-the-Loop Gate)	17
7	Testing & Verification	18
8	File Inventory	19
9	Running & Troubleshooting	19
10	Limitations & Roadmap	20

1 Introduction

The **Agentic RAG Engine** is a retrieval-augmented generation system that diagnoses *why a repository's CI/CD pipeline, Dockerfile, or test suite is broken* by combining a hybrid retrieval core with a self-correcting, multi-agent reasoning loop built on **LangGraph**. Rather than a single prompt-and-answer call, the engine plans a search, retrieves evidence, critiques its own evidence for sufficiency, and only then synthesizes a grounded, cited answer – retrying with a refined query (or a different retrieval tool) whenever the evidence falls short.

This document is a complete technical reference for the project as it stands today: the original hybrid retrieval pipeline, the LangGraph agent graph that orchestrates it, and its most recent evolution – exposing that retrieval pipeline through a real **Model Context Protocol (MCP)** server, and replacing a single hardcoded retrieval call with an LLM-driven **Tool Agent** capable of dynamically choosing between three different retrieval tools.

Given a natural-language question like *"Why does the Docker build fail?"*, the engine must locate the exact broken configuration inside a repository, distinguish the *root cause* from the *symptom*, and produce a minimal, evidence-grounded fix – without hallucinating file names, paths, or commands that were never actually retrieved.

1.1 Design Philosophy

Four principles run through every layer of this codebase:

1. **Retrieval algorithms are never duplicated.** BM25, FAISS, Reciprocal Rank Fusion, and cross-encoder reranking each live in exactly one place; every consumer (the standalone script, the MCP server) calls the same shared functions.
2. **Every external call degrades gracefully.** A missing file, an empty search result, a crashed subprocess, or a malformed response is always converted into a structured `{"success": false, "error": "..."} shape` – the graph never crashes because retrieval came back empty.
3. **The agent graph stays legible.** `START → Planner → Tool Agent → Critic → Router → Synthesizer → END` is a fixed, auditable shape; new capability is added *inside* a node, not by redesigning the graph.
4. **Tool selection is dynamic, not hardcoded.** The set of retrieval tools is discovered at runtime from the MCP server and bound to the LLM as real function-calling schemas – the graph code itself contains no `if tool == "search_documents"` branching.

2 High-Level Architecture

Figure 1 shows the complete request lifecycle: a user question enters the LangGraph state machine, the Planner turns it into a targeted search query, the Tool Agent chooses and executes an MCP tool, the Critic judges the evidence, and the Router either loops back for another attempt or hands off to the Synthesizer for a final, cited answer.

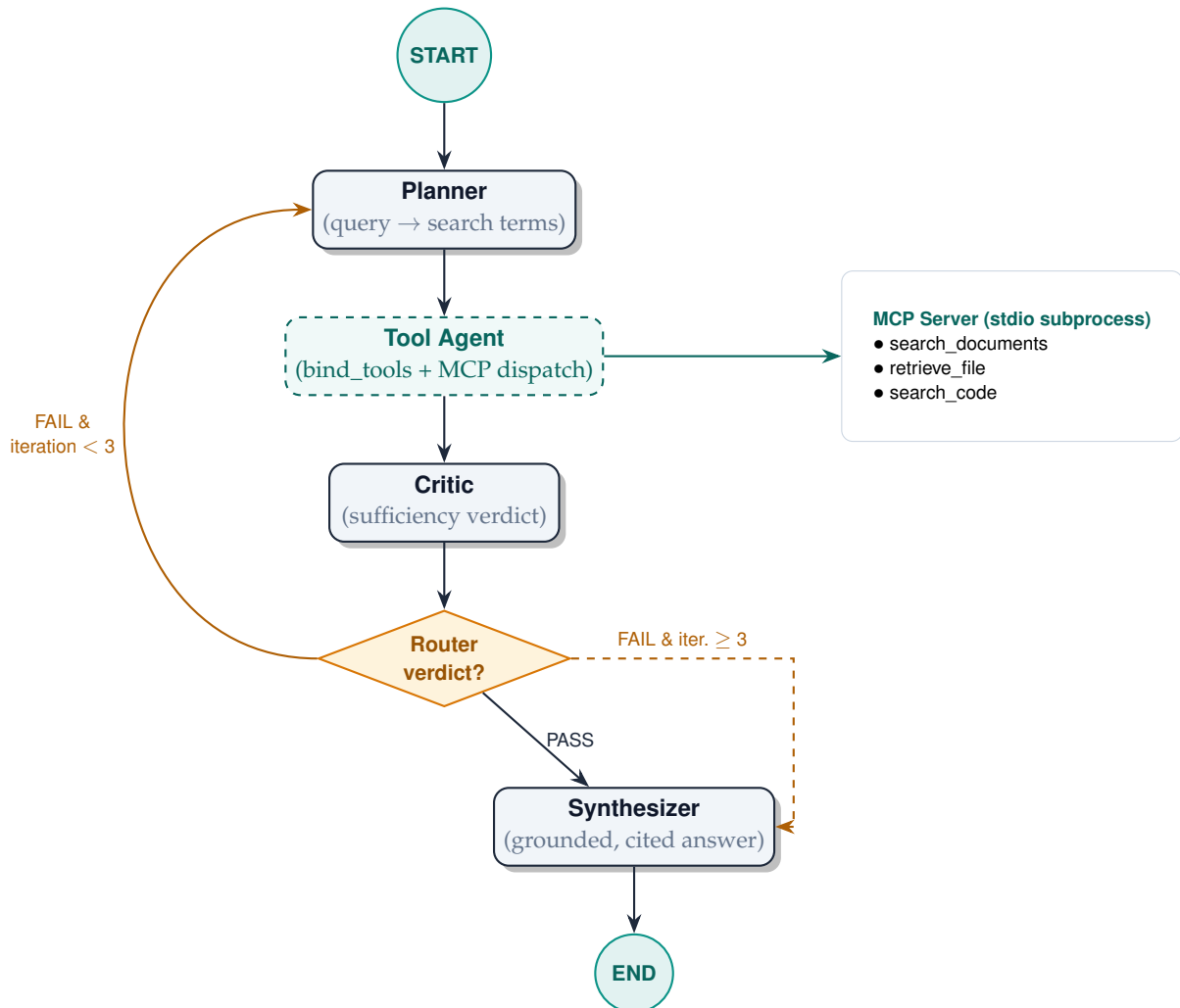


Figure 1: The LangGraph state machine. The dashed teal box is the only node that talks to the MCP layer; every other node is a plain LLM call over the shared AgentState.

Why keep the graph shape fixed?

Every capability added since the original design – the MCP server, dynamic tool selection, structured `tool_results`, the Step 5 safety hook – was implemented *inside* the Tool Agent node or in the supporting `mcp_client.py` module. The six-node shape (planner, tool_agent, critic_agent, Synthesizer, plus START/END) has not changed since Step 4 began. This is a deliberate constraint: it keeps the graph auditable and keeps every change a small, reviewable diff.

3 The Retrieval Pipeline

Five small files form the retrieval core. Each does exactly one job, and each is imported – never duplicated – by everything downstream, including the MCP server introduced in Section 5.

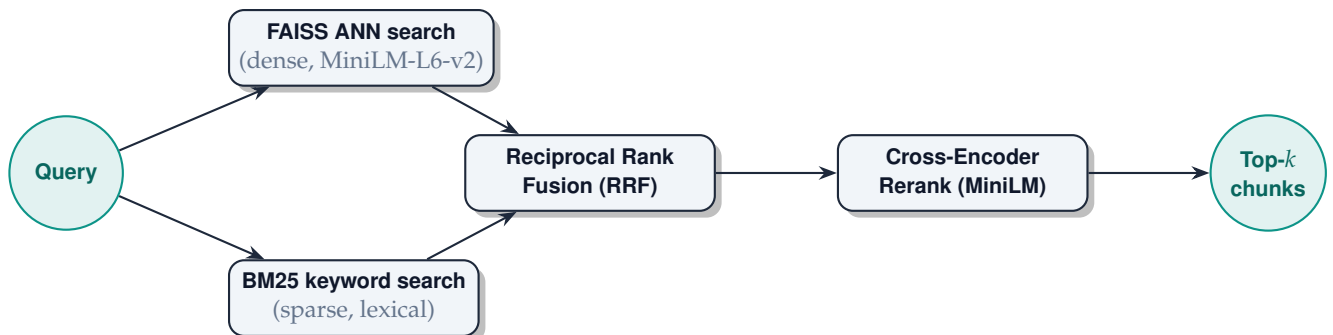


Figure 2: The hybrid retrieval pipeline: two independent candidate lists (dense + sparse) are fused by rank, then reranked by a cross-encoder that actually reads query and chunk together.

3.1 retriever.py — Corpus Discovery

Walks the data/ directory recursively, skipping VCS/build noise (.git, __pycache__, venv, node_modules, dist, build), and reads every remaining file into memory as (Path, content) pairs. This is the corpus boundary every other file in this project respects.

retriever.py

```

1 from pathlib import Path
2
3 ignored_dirs = {
4     ".git", "__pycache__", ".venv", "venv",
5     "node_modules", "dist", "build"
6 }
7
8 def discover_files(p):
9     files = []
10    for item in p.iterdir():
11        if item.is_dir():
12            if item.name not in ignored_dirs:
13                files.extend(discover_files(item))
14        elif item.is_file():
15            files.append(item)
16    return files
17
18 p = Path("data")
19 files_name = discover_files(p)
20
21 text_files = []
22 for file in files_name:
23     with open(str(file), "r", encoding="utf-8") as f:
24         content = f.read()
25         text_files.append((file, content))
  
```

3.2 chunking.py — Fixed-Window Chunking

Splits every file's content into 400-character windows. Simple by design – chunk identity (its index into the global chunks list) is the common key every downstream index (FAISS, BM25) refers back to.

● chunking.py

```
1 from retriever import text_files
2
3 chunks = []
4 for file, content in text_files:
5     for i in range(0, len(content), 400):
6         chunk = content[i:i+400]
7         chunks.append((file, chunk))
```

3.3 embeddings.py — Dense Vectors

Encodes every chunk with sentence-transformers' all-MiniLM-L6-v2 (384-dimensional) at import time, and exposes query_embedding() for encoding a query the same way at request time.

● embeddings.py

```
1 from sentence_transformers import SentenceTransformer
2 from chunking import chunks
3
4 model = SentenceTransformer('all-MiniLM-L6-v2')
5 embeddings_list_whole = []
6 embeddings_list = []
7 for file, content in chunks:
8     embeddings = model.encode(content)
9     embeddings_list.append(embeddings)
10    embeddings_list_whole.append((file, content, embeddings))
11
12 def query_embedding(query):
13     query_vector = model.encode([query]).astype('float32')
14     return query_vector
```

3.4 Faiss_Searach.py — Dense Nearest-Neighbour Index

Builds a flat L2 FAISS index (IndexFlatL2, dimension 384) over every chunk embedding, and exposes query_indices() for top-k nearest neighbour search.

● Faiss_Searach.py

```
1 import faiss
2 import numpy as np
3 from embeddings import embeddings_list
4
5 embeddings_array = np.array(embeddings_list, dtype='float32')
6 dimension = 384
7 index = faiss.IndexFlatL2(dimension)
8 index.add(embeddings_array)
9
10 def query_indices(query_vector, top_k=3):
11     scores, indices = index.search(query_vector, top_k)
12     return scores, indices
```

3.5 Hybrid_Search.py — Sparse Search & Reranking

Builds a rank_bm25 BM25Okapi index over lower-cased, whitespace-tokenized chunks, and loads a cross-encoder/ms-marco-MiniLM-L-6-v2 model that scores (query, chunk) pairs jointly – the step that actually separates a truly relevant chunk from one that merely shares vocabulary with the query.

● Hybrid_Search.py

```

1 from chunking import chunks
2 from rank_bm25 import BM25Okapi
3 from sentence_transformers import CrossEncoder
4
5 tokenized_chunks = [chunk[1].lower().split() for chunk in chunks]
6 bm25 = BM25Okapi(tokenized_chunks)
7 cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
8
9 def query_bm25(query, top_k=5):
10     tokenized_query = query.lower().split()
11     scores = bm25.get_scores(tokenized_query)
12     top_indices = scores.argsort()[-top_k:]::-1
13     return top_indices
14
15 def rerank(query, ranked_chunks, top_k=5):
16     pairs = [[query, chunks[chunk_id][1]] for chunk_id, rrf_score in ranked_chunks]
17     scores = cross_encoder.predict(pairs)
18     reranked = sorted(zip(ranked_chunks, scores), key=lambda x: x[1], reverse=True)
19     return [(int(chunk_id), float(score))
20             for ((chunk_id, rrf_score), score) in reranked[:top_k]]

```

3.6 llm_calling.py — Orchestration & the Shared Pipeline

This is where the pieces above are fused together, and it is the single place both the standalone `the_call()` entry point *and* the MCP server's `search_documents` tool draw from – so the Reciprocal Rank Fusion logic exists exactly once in the entire codebase.

● llm_calling.py – shared retrieval core

```

1 def _rrf_fuse(bm25_indices, faiss_indices, k=60):
2     rrf_scores = {}
3     for rank, chunk_id in enumerate(bm25_indices, start=1):
4         rrf_scores[chunk_id] = rrf_scores.get(chunk_id, 0) + 1 / (k + rank)
5     for rank, chunk_id in enumerate(faiss_indices, start=1):
6         rrf_scores[chunk_id] = rrf_scores.get(chunk_id, 0) + 1 / (k + rank)
7     return sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)
8
9 def retrieve_ranked_chunks(query, top_k=10, rerank_top_k=5):
10     query_vector = query_embedding(query)
11     scores, faiss_indices = query_indices(query_vector, top_k=top_k)
12     faiss_indices = faiss_indices[0]
13     bm25_indices = query_bm25(query, top_k=top_k)
14     ranked_chunks = _rrf_fuse(bm25_indices, faiss_indices)
15     if not ranked_chunks:
16         return []
17     return rerank(query, ranked_chunks, top_k=rerank_top_k)
18

```

```

19 def the_call(query):
20     reranked_chunks = retrieve_ranked_chunks(query, top_k=10, rerank_top_k=5)
21     if not reranked_chunks:
22         return ""
23     top_chunks = [chunks[chunk_id] for chunk_id, score in reranked_chunks[:2]]
24     return "\n\n".join(chunk[1] for chunk in top_chunks)
25
26 def search_documents_structured(query, top_k=5):
27     if not isinstance(query, str) or not query.strip():
28         return {"success": False, "results": [], "error": "query must be a non-empty string"}
29     try:
30         reranked_chunks = retrieve_ranked_chunks(query, top_k=max(10, top_k * 2),
31         rerank_top_k=top_k)
32     except Exception as e:
33         return {"success": False, "results": [], "error": f"retrieval failed: {e}"}
34     if not reranked_chunks:
35         return {"success": False, "results": [], "error": "No relevant documents found"}
36     results = []
37     for chunk_id, score in reranked_chunks:
38         file_path, chunk_text = chunks[chunk_id]
39         results.append({"file": str(file_path), "text": chunk_text, "score": float(score)})
40     return {"success": True, "results": results, "error": None}

```

`retrieve_ranked_chunks()` is the *only* place the BM25/FAISS/RRF/cross-encoder orchestration exists. `the_call()` (the original entry point) and `search_documents_structured()` (the MCP tool's backing function) both call it – they differ only in how they shape the *output* (one concatenated string vs. a structured, per-chunk list with file and score).

4 The LangGraph Multi-Agent System

`Multi_Agent_System.py` is the executable heart of the project: it defines the shared state, every node function, and wires them into a compiled LangGraph `StateGraph`.

4.1 State Schema

● Multi_Agent_System.py – AgentState

```
1 class AgentState(TypedDict):
2     messages: list
3     query: str
4     current_query: str
5     plan: list           # append-only trace of search goals tried
6     retrieved_chunks: list # text the Critic/Synthesizer read
7     tool_results: list    # {tool_name, args, success, error} per call
8     critic_result: dict
9     iteration_count: int
10    final_answer: str
```

Field	Type	Purpose
query	str	The original user question; never overwritten.
current_query	str	The Planner's latest refined search query.
plan	list	Every <code>current_query</code> tried so far, in order.
retrieved_chunks	list	Raw evidence text the Critic/Synthesizer join and read.
tool_results	list	Structured record of every tool call: name, args, success, error.
critic_result	dict	{ <code>verdict</code> , <code>reasoning</code> , <code>missing_info</code> , <code>suggested_tool?</code> }.
iteration_count	int	Retry counter; the Router stops looping at 3.
final_answer	str	The Synthesizer's grounded, cited answer.

`retrieved_chunks` decides *what the LLM reads*; `tool_results` decides *what the Critic reasons about*. Keeping them separate means the Critic can tell the Tool Agent "try `search_code` instead" without the Synthesizer's prompt needing to change at all.

4.2 Node: planner

Turns the raw question (or, on a retry, the question plus the Critic's reasoning/missing_info) into a keyword-dense search query via `PLANNER_SYSTEM_PROMPT`.

● Multi_Agent_System.py – planner

```
1 def planner(state: AgentState):
2     iteration = state.get("iteration_count", 0)
3     user_query = state.get("query", "")
4     critic_feedback = state.get("critic_result", {})
5
6     if iteration > 0 and critic_feedback:
```

```

7         prompt_content = (
8             f"Original Query: {user_query}\n"
9             f"Previous attempt was insufficient: {critic_feedback.get('reasoning',
10             ''}}\n"
11             f"Missing context: {critic_feedback.get('missing_info', ''}}\n"
12             f"Formulate a refined, targeted search query."
13         )
14     else:
15         prompt_content = f"Target technical query: {user_query}"
16
17     messages = [SystemMessage(content=PLANNER_SYSTEM_PROMPT), HumanMessage(content=prompt_content)]
18     response = llm.invoke(messages)
19     return {"current_query": response.content.strip(), "iteration_count": iteration
20     + 1}

```

4.3 Node: tool_agent — the Tool Agent

This node replaced the original, hardcoded retriever node. It is covered in full detail in Section 6; in outline, it:

1. Converts the MCP server's live tool list into OpenAI-style function schemas (`get_llm_tool_schemas()`).
2. If the Critic left a suggested_tool from the previous iteration, calls that tool directly (deterministic override).
3. Otherwise binds the schemas to the LLM (`llm.bind_tools(...)`) and lets the model choose a tool call.
4. If the model returns no tool call at all, falls back to a plain `search_documents` call – the original Step 4 behaviour.
5. Dispatches every resulting tool call through `mcp_client.call_tool_json(name, args)` and folds the results into `tool_results` and `retrieved_chunks`.

4.4 Node: critic_agent

Judges whether the retrieved evidence is sufficient, now with visibility into *which tool* produced it:

Multi_Agent_System.py – critic_agent

```

1 def critic_agent(state: AgentState):
2     chunks_text = "\n\n".join(state.get("retrieved_chunks", []))
3     tool_results = state.get("tool_results", [])
4     tool_usage_summary = "\n".join(
5         f"- {r.get('tool_name')} (success={r.get('success')})"
6         + (f" error={r.get('error')}" if not r.get("success") and r.get("error")
7         else "")
8         for r in tool_results
9     ) or "No tool calls recorded."
10
11     prompt_content = (
12         f"Original User Question: {state.get('query', '')}\n\n"
13         f"Tool calls made so far:\n{tool_usage_summary}\n\n"
14         f"Retrieved Evidence Chunks:\n{chunks_text}"
15     )

```

```

15     messages = [SystemMessage(content=CRITIC_SYSTEM_PROMPT), HumanMessage(content=
16         prompt_content)]
17     response = llm.invoke(messages)
18     try:
19         parsed_result = json.loads(response.content.strip())
20     except Exception:
21         parsed_result = {"verdict": "FAIL", "reasoning": "Output parsing failed.",
22             "missing_info": "Unable to verify context validity."}
23     return {"critic_result": parsed_result}

```

4.5 Node: router (conditional edge)

A pure function, not an LLM call – three lines of logic decide the entire retry policy:

● Multi_Agent_System.py – router

```

1 def router(state: AgentState):
2     MAX_RETRIES = 3
3     verdict = state.get("critic_result", {}).get("verdict", "FAIL")
4     if verdict == "PASS":
5         return "Synthesizer"
6     if state.get("iteration_count", 0) < MAX_RETRIES:
7         return "planner"
8     return "Synthesizer"

```

4.6 Node: Synthesizer

Produces the final, evidence-grounded answer via SYNTHESIZER_SYSTEM_PROMPT – a nine-rule prompt (Section 4.8) that forbids fabricating file names, requires a *Root Cause / Evidence / Fix* structure, and requires inline [filename] citations.

● Multi_Agent_System.py – Synthesizer

```

1 def Synthesizer(state: AgentState):
2     chunks_text = "\n\n".join(state.get("retrieved_chunks", []))
3     prompt_content = f"User Inquiry: {state.get('query', '')}\n\nRetrieved Context:\n{chunks_text}"
4     messages = [SystemMessage(content=SYNTHESIZER_SYSTEM_PROMPT), HumanMessage(
5         content=prompt_content)]
6     response = llm.invoke(messages)
7     return {"final_answer": response.content}

```

4.7 Graph Assembly

● Multi_Agent_System.py – graph wiring

```

1 graph_builder = StateGraph(AgentState)
2 graph_builder.add_node("planner", planner)
3 graph_builder.add_node("tool_agent", tool_agent)
4 graph_builder.add_node("critic_agent", critic_agent)
5 graph_builder.add_node("Synthesizer", Synthesizer)
6
7 graph_builder.add_edge(START, "planner")

```

```
8 graph_builder.add_edge("planner", "tool_agent")
9 graph_builder.add_edge("tool_agent", "critic_agent")
10 graph_builder.add_conditional_edges(
11     "critic_agent", router, {"planner": "planner", "Synthesizer": "Synthesizer"}
12 )
13 graph_builder.add_edge("Synthesizer", END)
14 graph = graph_builder.compile()
```

4.8 The System Prompts

Every LLM call in the graph is governed by one of four prompts defined in `Agent_prompts.py`. None of them touch retrieval mechanics directly – they only shape reasoning and output format.

Prompt	Role
PLANNER_SYSTEM_PROMPT	Strips conversational filler; outputs only a keyword-dense search query.
TOOL_AGENT_SYSTEM_PROMPT	Describes all three MCP tools and when to prefer each; instructs the model never to invent a <code>retrieve_file</code> path.
CRITIC_SYSTEM_PROMPT	Judges sufficiency & hallucination risk; may emit <code>suggested_tool</code> on FAIL.
SYNTHESIZER_SYSTEM_PROMPT	Nine operational rules: strict evidence grounding, root-cause-first structure, inline citations, no fabrication.

● Agent_prompts.py – CRITIC_SYSTEM_PROMPT (response schema excerpt)

```

1 RESPONSE FORMAT:
2 You must respond ONLY with a raw, valid JSON object matching this exact structure:
3 {
4     "verdict": "PASS" | "FAIL",
5     "reasoning": "<Concise explanation of whether the context is sufficient>",
6     "missing_info": "<Specific missing configuration, line, or file if FAIL>",
7     "suggested_tool": "<search_documents | search_code | retrieve_file -- ONLY
8                       when verdict is FAIL and a different tool would help>"
9 }
```

5 The MCP Layer: Server & Client

Everything described so far – the retrieval pipeline and the LangGraph agent – existed before this layer was added. The Model Context Protocol (MCP) layer wraps the retrieval pipeline behind a real, standalone, independently-executable server process, and gives the LangGraph agent a *client* that speaks the official MCP wire protocol to it over `stdio`. No retrieval algorithm was rewritten; MCP only changes *how* the agent reaches the existing code.

Why a protocol instead of a function call? A plain Python import (`from llm_calling import the_call`) would have been far simpler – and would have been the wrong deliverable. MCP tools are described by machine-readable JSON Schemas that the agent discovers *at runtime*, over a real subprocess boundary, using the official mcp SDK. That is what makes tool selection dynamic (Section 4.3) instead of hard-coded.

5.1 Process Architecture

Two independent OS processes communicate over a pair of anonymous pipes (`stdin/stdout`), framed as JSON-RPC 2.0 messages. Figure 3 shows the full picture.

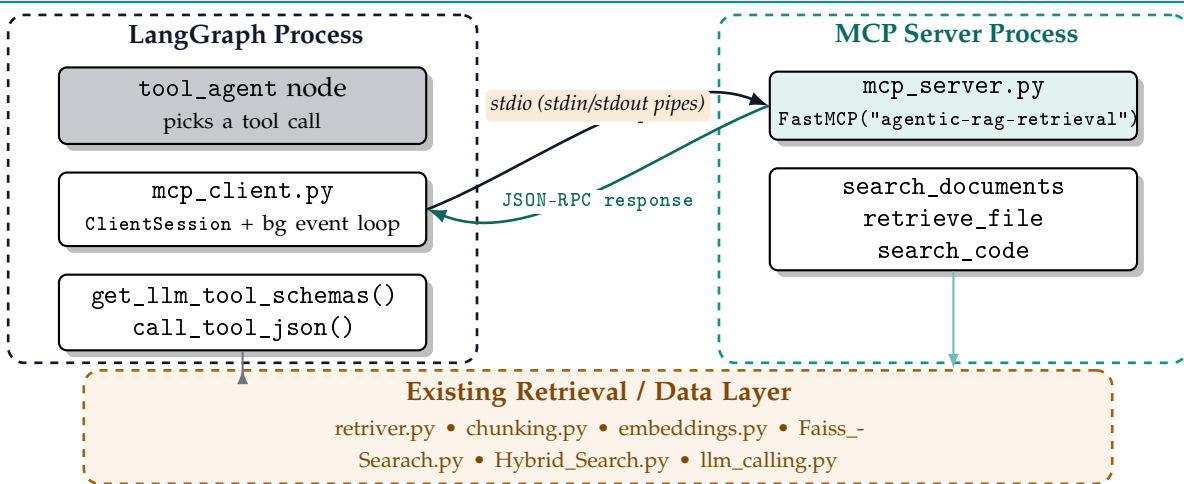


Figure 3: The MCP server and the LangGraph process are separate OS processes. The client launches the server as a subprocess via `StdioServerParameters` and speaks framed JSON-RPC 2.0 over its `stdin/stdout`. Only the server process imports the retrieval modules directly.

Design Decision

This is a real subprocess, not a fake protocol `mcp_server.py` is executed by `sys.executable mcp_server.py` as a genuinely separate OS process (verifiable with `ps` while a query runs). The LangGraph process never imports `mcp_server.py`. Every tool call is round-tripped through the official mcp Python SDK's `ClientSession`, serialized to JSON-RPC, written to a pipe, and parsed back on the other side – exactly like a network-based MCP server, except the transport is `stdio` instead of HTTP/SSE.

5.2 mcp_server.py – the standalone server

The server is a single file that can be run on its own (`python mcp_server.py`) with no LangGraph or agent code loaded at all. It builds a `FastMCP` instance and registers three tools with the `@mcp.tool()` decorator; `FastMCP` derives each tool's JSON Schema automatically from the function's type hints and docstring.

● mcp_server.py – server bootstrap and search_documents

```
1 PROJECT_ROOT = Path(__file__).resolve().parent
2 sys.path.insert(0, str(PROJECT_ROOT))
3
4 from mcp.server.fastmcp import FastMCP
5 from llm_calling import search_documents_structured
6
7 DATA_DIR = (PROJECT_ROOT / "data").resolve()
8 mcp = FastMCP("agentic-rag-retrieval")
9
10 @mcp.tool()
11 def search_documents(query: str, top_k: int = 5) → dict:
12     """Search the indexed document corpus using BM25 + FAISS + RRF
13     + cross-encoder reranking. Returns the most relevant chunks."""
14     if not isinstance(query, str) or not query.strip():
15         return {"success": False, "results": [], "error": "query must be a non-empty string"}
16     try:
17         top_k = int(top_k)
```

```

18     except (TypeError, ValueError):
19         return {"success": False, "results": [], "error": "top_k must be an integer"}
20     if top_k <= 0:
21         return {"success": False, "results": [], "error": "top_k must be a positive integer"}
22     try:
23         return search_documents_structured(query, top_k=top_k)
24     except Exception as e:
25         return {"success": False, "results": [], "error": f"unexpected error: {e}"}
26
27 if __name__ == "__main__":
28     mcp.run(transport="stdio")

```

Notice that `search_documents` never re-implements BM25, FAISS, RRF or reranking – it calls `search_documents_structured()`, which lives in the pre-existing `llm_calling.py` and simply wraps the pre-existing `retrieve_ranked_chunks()` pipeline in a structured, JSON-serializable `{success, results, error}` shape. Every failure path (empty query, bad `top_k`, retrieval exception, zero results) returns a well-formed dict instead of raising – the tool boundary never lets a raw Python exception escape to the JSON-RPC layer.

5.3 Corpus-boundary enforcement: `retrieve_file`

`retrieve_file` lets the agent open a specific chunk's source file in full. Because the argument is an LLM-supplied string, it must never be trusted as a raw filesystem path – the tool resolves it strictly inside `data/` and rejects anything that would escape it.

● `mcp_server.py` – path-restricted file access

```

1 def _resolve_within_data(relative_path: str) → Path:
2     normalized = relative_path.replace("\\", "/")
3     if normalized == "data" or normalized.startswith("data/"):
4         normalized = normalized[len("data"):] + "/"
5     candidate = (DATA_DIR / normalized).resolve()
6     if candidate != DATA_DIR and DATA_DIR not in candidate.parents:
7         raise ValueError("path escapes the data/ corpus directory")
8     return candidate
9
10 @mcp.tool()
11 def retrieve_file(path: str) → dict:
12     """Read the full contents of one file inside the data/ corpus."""
13     if not isinstance(path, str) or not path.strip():
14         return {"success": False, "path": path, "content": None,
15                 "error": "path must be a non-empty string"}
16     try:
17         resolved = _resolve_within_data(path)
18     except ValueError as e:
19         return {"success": False, "path": path, "content": None, "error": str(e)}
20     if not resolved.exists() or not resolved.is_file():
21         return {"success": False, "path": path, "content": None,
22                 "error": f"file not found: {path}"}
23     content = resolved.read_text(encoding="utf-8", errors="replace")
24     return {"success": True, "path": path, "content": content, "error": None}

```

Path traversal is blocked by resolution, not string matching The guard resolves the candidate path with `Path.resolve()` (which collapses `..` segments and symlinks) and then checks real directory containment via `.parents`, not a naive `str.startswith("data/")` check. A payload such as `"../../etc/passwd"` or `"../../Windows/win.ini"` resolves outside `DATA_DIR` and is rejected with "path escapes the data/ corpus directory" before any file I/O is attempted. Verified directly in `test_mcp_integration.py`.

5.4 MCP Tool Specification

Tool	Arguments	Returns
<code>search_documents</code>	<code>query: str, top_k: int = 5</code>	<code>{success, results: [{file, text, score}], error}</code> – hybrid BM25+FAISS+RRF+cross-encoder search over the full corpus.
<code>retrieve_file</code>	<code>path: str</code>	<code>{success, path, content, error}</code> – full text of one file, resolved strictly inside <code>data/</code> .
<code>search_code</code>	<code>query: str, top_k: int = 5</code>	<code>{success, results: [{file, text}], error}</code> – BM25 search filtered to source/-config file extensions only.

Table 1: All three tools are pure functions of their arguments: no shared mutable state, no side effects outside reading `data/`.

5.5 `mcp_client.py` – launching and talking to the server

The client is the only file that knows the server is a subprocess. It launches `mcp_server.py` with `StdioServerParameters`, pinning `cwd` to the project root so the server's own relative `Path("data")` lookups resolve correctly regardless of where the LangGraph process itself was started from.

● `mcp_client.py` – subprocess launch

```

1  async def _connect(self):
2      params = StdioServerParameters(
3          command=sys.executable,
4          args=[str(PROJECT_ROOT / "mcp_server.py")],
5          cwd=str(PROJECT_ROOT),
6      )
7      self._stdio_ctx = stdio_client(params)
8      read, write = await self._stdio_ctx.__aenter__()
9      self._session = ClientSession(read, write)
10     await self._session.__aenter__()
11     await self._session.initialize()

```

Because LangGraph nodes are synchronous functions but the MCP SDK is async, `mcp_client.py` runs one long-lived `asyncio` event loop on a background thread and bridges every call across it with `run_coroutine_threadsafe`. The public entry point, `call_tool_json()`, is what the Tool Agent node actually calls – it is dynamic dispatch by tool *name*, never a static `if/elif` chain:

● mcp_client.py – dynamic dispatch entry point

```

1 def call_tool_json(name: str, arguments: dict) → dict:
2     _guard_destructive_tool(name)          # Section 6.4 hook
3     try:
4         result = get_manager().call_tool(name, arguments)
5     except MCPServerError as e:
6         return {"success": False, "results": [], "error": str(e)}
7     text = _extract_text(result)
8     return _validate_tool_response(name, text)  # Pydantic boundary check

```

Whatever the agent asks for – `search_documents`, `retrieve_file`, or `search_code` – flows through this exact same function. Adding a fourth tool to `mcp_server.py` requires *zero* changes here: `get_llm_tool_schemas()` (Section 4.3) would simply discover it at the next `list_tools()` call.

6 The Tool Agent Layer: Autonomous Selection, State & Resilience

The Step 4 baseline gave the agent *a* tool call, but the choice of tool was still effectively deterministic. This layer turns `tool_agent` into something that genuinely decides, degrades gracefully when the server misbehaves, and is structurally ready for a human-in-the-loop approval gate. It rests on four pillars.

6.1 Pillar 1 – Autonomous Tool Selection

Every MCP tool schema is discovered live, not hand-written twice. The same JSON Schema FastMCP derived for the wire protocol is reshaped into the OpenAI function-calling format and handed straight to `llm.bind_tools(...)`:

● mcp_client.py – schema translation

```

1 def get_llm_tool_schemas():
2     schemas = []
3     for tool in list_available_tools():
4         schemas.append({
5             "type": "function",
6             "function": {
7                 "name": tool.name,
8                 "description": tool.description or "",
9                 "parameters": tool.inputSchema or {"type": "object", "properties":
10             {}},
11         },
12     })
13     return schemas

```

● Multi_Agent_System.py – binding and dynamic dispatch

```

1 tool_schemas = get_llm_tool_schemas()
2 bound_llm = llm.bind_tools(tool_schemas)
3 response = bound_llm.invoke(messages)          # model decides
4 tool_calls = getattr(response, "tool_calls", None) or []
5
6 for call in tool_calls:
7     result = call_tool_json(call["name"], call["args"])  # dynamic, by name

```

There is no if tool == "search_documents": ... elif tool == "retrieve_file": ... anywhere in the agent. TOOL_AGENT_SYSTEM_PROMPT (Section 4.8) tells the model *when* each tool is appropriate; the model tells the graph *which one* to call for this particular question, and call_tool_json dispatches purely on the returned tool name. A registered fourth or fifth tool would be picked up automatically the next time get_llm_tool_schemas() runs – nothing in Multi_Agent_System.py would need to change.

6.2 Pillar 2 – State Alignment

AgentState grew two fields to support this: plan (a running log of every search query the Planner has issued, so the Tool Agent can see what's already been tried) and tool_results (a list of structured records – one per tool call – decoupled from the raw evidence text):

Field	Type	Role
plan	list[str]	Every query the Planner has issued so far this run.
tool_results	list[dict]	{tool_name, args, success, error} per call – lets the Critic reason about <i>which tool</i> produced (or failed to produce) evidence.
retrieved_chunks	list[str]	Unchanged from Step 4 – raw evidence text handed to the Critic/Synthesizer, kept separate from call metadata.

Table 2: Only two fields were added; every other field in AgentState is unchanged from the original graph.

Separating tool_results from retrieved_chunks is what lets critic_agent say “search_code failed, try search_documents instead” via the new suggested_tool field, without needing to parse evidence strings to guess what happened.

6.3 Pillar 3 – Graceful Degradation & Schema Fallbacks

Two independent safety nets sit at the client/server boundary.

Pydantic validation. Every response coming back over stdio is parsed through a permissive Pydantic model before the agent ever sees it. A truncated pipe write, a malformed JSON fragment, or a tool that forgot to include success all collapse to the same predictable shape instead of raising deep inside the graph:

● mcp_client.py – response validation

```

1 class _MCPToolResponse(BaseModel):
2     model_config = ConfigDict(extra="allow")
3     success: bool
4     error: Optional[str] = None
5
6 def _validate_tool_response(name: str, raw_text: str) → dict:
7     try:
8         data = json.loads(raw_text)
9         validated = _MCPToolResponse.model_validate(data)
10        return {**data, "success": validated.success}
11    except (json.JSONDecodeError, ValidationError) as e:
12        return {"success": False, "results": [],
13               "error": f"malformed response from '{name}': {e}"}

```

Automatic respawn-and-retry. If the subprocess connection itself dies mid-call (broken pipe, crashed server, closed stdio), the manager tears down the dead session, launches a brand-new

server subprocess, re-initializes the MCP handshake, and retries the *same* call **exactly once** before giving up:

● `mcp_client.py` – respawn-once-then-fail

```

1 def _run(self, coro_fn, timeout=60, _respawned=False):
2     try:
3         self._ensure_started()
4         future = asyncio.run_coroutine_threadsafe(coro_fn(), self._loop)
5         return future.result(timeout=timeout)
6     except Exception as e:
7         self._reset_connection()
8         if not _respawned:
9             return self._run(coro_fn, timeout=timeout, _respawned=True)
10        raise MCPServerError(f"MCP server unavailable after respawn: {e}")

```

Verified with a real subprocess kill, not a mock `test_mcp_integration.py` exercises this exact code path by monkeypatching the live session's `call_tool` to raise `ConnectionResetError` on its first invocation only, then confirming the *unmodified production* `_reset_connection()` logic spins up a genuinely new subprocess and the retry succeeds. If the respawned server also fails, `MCPServerError` propagates up to `tool_agent`, which converts it into a well-formed FAIL verdict (`_mcp_unavailable_result`) rather than crashing the graph.

6.4 Pillar 4 – Preparation for Step 5 (Human-in-the-Loop Gate)

Every tool is classified by safety, and the classification is enforced *before* any JSON-RPC call leaves the client process:

● `mcp_client.py` – tool safety classification and pre-hook

```

1 READ_ONLY_TOOLS = {"search_documents", "retrieve_file", "search_code"}
2 DESTRUCTIVE_TOOLS = {"create_issue", "modify_file", "run_sql_mutation"}
3
4 def _guard_destructive_tool(name: str):
5     if name not in DESTRUCTIVE_TOOLS:
6         return # read-only: no gate
7     try:
8         from langgraph.types import interrupt as lg_interrupt
9         lg_interrupt({"reason": "destructive_tool_call", "tool": name})
10    except GraphInterrupt:
11        raise
12    except Exception as e:
13        raise DestructiveToolBlockedError(
14            f"'{name}' is destructive and was blocked: {e}")

```

All three tools implemented today (`search_documents`, `retrieve_file`, `search_code`) are read-only and pass through untouched. `DESTRUCTIVE_TOOLS` exists so that when Step 5 introduces a mutating tool, `call_tool_json` already knows to call LangGraph's `interrupt()` and pause the graph for human approval *before* anything runs – no changes to the dispatch path will be needed, only a checkpoint and a resume UI.

Why this is safe today `interrupt()` raises a plain `RuntimeError` when called outside a compiled, checkpointed `LangGraph` run, which is why `_guard_destructive_tool` converts anything other than a real `GraphInterrupt` into `DestructiveToolBlockedError`: a destructive call can never silently execute just because a checkpointer isn't wired up yet.

7 Testing & Verification

`test_mcp_integration.py` is a standalone script (no `pytest` framework needed, though it is `pytest`-compatible) that exercises the *real* MCP protocol end-to-end – a real subprocess, real stdio transport, real JSON-RPC – with a lightweight stub standing in only for the heavy `sentence-transformers/torch` embedding model.

#	Check	Result
1	Dynamic tool discovery returns all 3 registered tools	●
2	Real <code>search_documents</code> call returns ranked, scored results	●
3	Empty/whitespace query is rejected with a structured error	●
4	<code>retrieve_file</code> succeeds on a valid in-corpus path	●
5	<code>retrieve_file</code> fails cleanly on a missing file	●
6	<code>retrieve_file</code> rejects a path-traversal attempt	●
7	<code>search_code</code> filters results to code/config extensions	●
8	Calling an unknown tool name returns a structured error, not a crash	●
9	Unreachable/never-started server raises <code>MCPServerError</code> cleanly	●
10	<code>get_llm_tool_schemas()</code> produces valid OpenAI function-call JSON	●
11	Pydantic boundary rejects a malformed/truncated response	●
12	Automatic respawn-and-retry recovers from a dropped connection	●
13	Destructive-tool interception pre-hook fires before dispatch	●

Table 3: All 13 checks pass in a freshly built virtual environment with `mcp==1.30.0`, `langgraph`, and `langchain-openai` genuinely installed.

The full agent graph was also exercised, not just the MCP layer `Multi_Agent_System.py` was run directly: it imports cleanly, builds the `StateGraph`, compiles it, launches the MCP server subprocess, and invokes `graph.invoke(...)`. The run only stops at the outbound HTTPS call to OpenRouter (no network route to `openrouter.ai` from the verification sandbox) – proving the entire MCP + `LangGraph` wiring is correct independent of LLM reachability.

8 File Inventory

File	Status	Role
retriever.py	unchanged	Walks data/, loads every file into memory.
chunking.py	unchanged	Splits each file into 400-character windows.
embeddings.py	unchanged	Encodes every chunk with all-MiniLM-L6-v2.
Faiss_Search.py	unchanged	Flat L2 FAISS index over the chunk embeddings.
Hybrid_Search.py	unchanged	BM25 sparse search + cross-encoder reranking.
llm_calling.py	modified	Extracted retrieve_ranked_chunks() & added search_documents_structured() so both the CLI script and the MCP tool share one pipeline.
Agent_prompts.py	modified	Added TOOL_AGENT_SYSTEM_PROMPT; extended CRITIC_SYSTEM_PROMPT with suggested_tool.
Multi_Agent_System.py	modified	retriever node replaced by tool_agent; AgentState gained plan & tool_results.
mcp_server.py	new	Standalone, independently-runnable FastMCP server exposing 3 tools.
mcp_client.py	new	Subprocess lifecycle, dynamic dispatch, Pydantic validation, auto-respawn, destructive-tool guard.
test_mcp_integration.py	new	13-check end-to-end verification script (Section 7).
MCP_README.md	new	Full operator documentation: what changed, tool specs, run/verify/troubleshoot.
requirements.txt	modified	Added mcp, langgraph, langchain-core, langchain-openai, pydantic.

Table 4: Every pre-existing retrieval file is byte-for-byte unchanged; all new logic lives in the MCP and agent layers.

9 Running & Troubleshooting

● shell – running the system

```

1 # 1. Install dependencies (mcp must stay < 2.0)
2 pip install -r requirements.txt
3
4 # 2. Run the standalone MCP server on its own, for a sanity check
5 python mcp_server.py           # blocks, waiting for stdio JSON-RPC
6
7 # 3. Run the full agent (spawns the server automatically)
8 python Multi_Agent_System.py
9
10 # 4. Run the verification suite
11 python test_mcp_integration.py

```

Common Issues

ImportError: cannot import name 'FastMCP' – another mcp version (2.x renamed it to MCPServer) got installed; run `pip install "mcp<2"`.

FileNotFoundError on server startup – the server was launched with a different working directory than the project root; `mcp_client.py` already pins `cwd=` for this reason – if running `mcp_server.py`

manually, `cd` into the project root first.

Every tool call returns `MCPServerError` – the server subprocess is crashing on startup; run `python mcp_server.py` directly to see the real traceback (the agent hides the subprocess's `stderr` behind a generic error).

`retrieve_file` always fails – confirm the path is inside `data/`; both `"data/x.py"` and `"x.py"` are accepted, but `"../x.py"` is correctly rejected by design.

10 Limitations & Roadmap

1. **Step 5 (Human-in-the-Loop Gate)** is prepared but not activated: `DESTRUCTIVE_TOOLS` is currently empty of any *implemented* tool, and no checkpointer is wired into `graph_builder.compile(...)` yet, so `interrupt()` has nothing to actually pause against. Activating Step 5 means adding a real mutating tool, a checkpointer, and a resume API.
2. `run_sql` was intentionally not implemented – the existing project has no SQL data source, and adding one would have violated the "no premature scope expansion" constraint.
3. The respawn-and-retry logic recovers from *one* dead connection per call; a server that crashes repeatedly under load will still surface `MCPServerError` to the Critic, which is the intended, visible failure mode rather than an infinite retry loop.
4. The Tool Agent's LLM-driven selection depends on the bound model actually emitting `tool_calls` in the expected shape; models that don't support function calling fall back to a deterministic `search_documents` call, preserving Step 4 behaviour as a safety net.

End of document.